

第 8 章：Actor-Critic：从 Monte Carlo Baseline 到 TD、GAE 与 A2C/A3C

8.1 本章符号说明

符号/缩写	English	中文	含义
AC	Actor-Critic	行动者-评价者框架	Actor 学策略，Critic 学价值并给 Actor 提供学习信号。
A2C	Advantage Actor-Critic	优势 Actor-Critic	多环境同步采样、统一更新的 on-policy Actor-Critic 算法。
A3C	Asynchronous Advantage Actor-Critic	异步优势 Actor-Critic	多 worker 异步采样并更新共享参数的算法。
GAE	Generalized Advantage Estimation	广义优势估计	对多步 TD residual 加权，构造 advantage estimate。
TD	Temporal Difference	时序差分	使用 reward 加下一状态价值进行 bootstrap 的估计方法。
MC	Monte Carlo	蒙特卡洛	使用完整实际 return、不 bootstrap 的估计方法。
$S_t / A_t / R_{t+1}$	State / Action / Reward Random Variable	状态 / 动作 / 奖励随机变量	时刻 t 的状态、动作和执行动作后收到的奖励。
$\pi_\theta(a s)$	Actor Policy	Actor 策略	参数为 θ 的动作分布。
θ	Actor Parameters	Actor 参数	策略网络的可学习参数。
$V_w(s)$	Critic State-value Estimate	Critic 状态价值估计	参数为 w 的价值网络输出。
w	Critic Parameters	Critic 参数	价值网络的可学习参数。
$V_\pi(s) / Q_\pi(s,a)$	True Value Functions	理论价值函数	策略 π 下的真实条件期望，用于推导。
$A_\pi(s,a)$	Advantage Function	理论优势函数	$Q_\pi(s,a) - V_\pi(s)$ 。
$\hat{A}_t$	Advantage Estimate	优势估计量	实际更新 Actor 时使用的带噪声估计；帽子表示它不是真实 A_π 。
δ_t	TD Residual / TD Error	TD 残差 / 时序差分误差	一步 bootstrap target 与当前 $V_w(s_t)$ 的差。
$Y_t^{(1)}$	One-step Value Target	一步价值目标	$R_{t+1} + \gamma(1-d_t)V_w(S_{t+1})$ 。
$Y_t^{(n)}$	n-step Value Target	n 步价值目标	连续 n 步实际 reward 加第 n 步 bootstrap value。
d_t	Terminal Indicator	终止指示量	真正终止时为 1，此后没有环境定义的未来价值。
γ	Discount Factor	折扣因子	控制未来 reward/value 的权重。
λ	GAE Lambda	GAE 衰减参数	控制 advantage estimate 看多远。
L_{actor} / L_{critic}	Actor / Critic Loss	Actor / Critic 损失	分别更新策略和价值网络。

符号/缩写	English	中文	含义
$H(\pi)$	Policy Entropy	策略熵	衡量动作分布随机程度。
β_H	Entropy Coefficient	熵系数	控制 entropy bonus 的强度。
c_V	Value-loss Coefficient	价值损失系数	共享总 loss 中 Critic loss 的权重。
$sg(\cdot)$	Stop Gradient	停止梯度	前向使用数值，反向不沿该路径传播梯度。
K_t / U_t	Discrete Type / Continuous Parameter	离散类型 / 连续参数随机变量	混合动作 $A_t = (K_t U_t)$ 的两个组成部分。
N / T	Number of Environments / Rollout Length	并行环境数 / Rollout 长度	A2C 中同步采样的环境数量和每个环境采样的步数。
on-policy	On-policy Learning	同策略学习	Rollout 由当前正在更新的策略产生。

8.2 本章目标

学完本章，你应该能说明：

1. 第 7 章的 learned baseline 为什么还不是本章意义下的 Critic。
2. Critic 怎样通过 bootstrap 从 Monte Carlo baseline 过渡到 TD value learning。
3. 为什么一步 TD residual 的条件期望等于 advantage。
4. n-step advantage 和 GAE 怎样在 bias 与 variance 之间折中。
5. Actor loss、Critic loss、stop gradient 和 entropy bonus 分别做什么。
6. A2C/A3C 是怎样组织采样与更新的，而不是把它们当成突然出现的新公式。

8.3 本章主线与章节边界

第 7 章停在：

$$\hat{A}_t^{MC} = G_t - b_\phi(s_t)$$

它已经能降低方差，但仍要等完整回报 G_t 。本章按下面顺序解决这个等待和高方差问题：

```

REINFORCE with learned baseline
-> baseline 用完整 G_t 回归，仍是 Monte Carlo
-> 允许 Value estimator 用下一状态价值 bootstrap
-> 得到一步 TD Critic
-> TD residual 可估计 advantage
-> 用 n-step 在一步 TD 与完整 MC 之间折中
-> 用 GAE 平滑组合多个时间尺度
-> 写出 Actor/Critic/Entropy 三类 loss
-> A2C/A3C 只是采样和更新的组织方式
    
```

本章不处理“策略一步更新太大”的问题；那是第 9 章 TRPO/PPO 的主题。

8.4 从 Learned Baseline 到 Critic

8.4.1 第 7 章 Baseline 的限制

REINFORCE with baseline 训练：

$$L_b(w) = \mathbb{E}_\pi[(V_w(s_t) - G_t)^2]$$

这里把 baseline 直接记为 V_w ，因为它试图近似 V_π 。但监督标签 G_t 只有等未来轨迹走完后才能得到。

优点：

- 标签不 bootstrap，估计目标直接来自真实轨迹。
- 在样本足够时，对当前策略价值的估计偏差较小。

问题：

- G_t 方差高。
- 长 episode 要等很久才能更新。
- 每条轨迹只能提供一次具体未来，样本效率较低。

8.4.2 Critic 的关键变化是 Bootstrap

由 Bellman expectation：

$$V_\pi(S_t) = \mathbb{E}_\pi [R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = S_t]$$

Critic 用当前价值估计构造一步 target：

$$Y_t^{(l)} = R_{t+1} + \gamma(1-d_t)V_w(S_{t+1})$$

再训练：

$$L_{critic}(w) = \mathbb{E}_t [(V_w(S_t) - sg(Y_t^{(l)}))^2]$$

这就是 bootstrap：target 的一部分来自模型自己的下一状态预测，而不是全部等待真实未来奖励。

它带来明确交换：

目标	等待时间	方差	估计偏差
完整 G_t	长	高	不含 bootstrap bias
一步 $Y_t^{(l)}$	一步	较低	受 Critic 误差影响

8.4.3 为什么 Target 要 Stop Gradient

更新当前 $V_w(S_t)$ 时， $Y_t^{(l)}$ 被当作监督目标：

$$sg(Y_t^{(l)})$$

如果不 stop gradient，优化器可能同时改变当前预测和 target 中的下一状态预测，目标会跟着被拟合对象一起移动。基础 TD Critic 通常只对 $V_w(S_t)$ 这一侧求梯度。

8.5 一步 Actor-Critic

8.5.1 TD Residual

一步 TD residual 定义为：

$$\delta_t = R_{t+1} + \gamma(1-d_t)V_w(S_{t+1}) - V_w(S_t)$$

它回答：

执行动作后的即时奖励加下一状态估值，是否超过了行动前对当前状态的预期？

- $\delta_t > 0$: 结果比当前状态平均预期好。
- $\delta_t < 0$: 结果比预期差。

8.5.2 为什么 δ_t 可以估计 Advantage

理论 advantage:

$$A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$$

动作价值可写成:

$$Q_{\pi}(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(1-d_t)V_{\pi}(S_{t+1}) \mid s_t, a_t]$$

代入:

$$A_{\pi}(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(1-d_t)V_{\pi}(S_{t+1}) - V_{\pi}(s_t) \mid s_t, a_t]$$

如果 $V_w \approx V_{\pi}$, 并用一次 transition 近似条件期望:

$$\hat{A}_t^{(l)} = \delta_t$$

必须保留两个限定:

1. δ_t 是一次采样的 advantage estimate, 不是真实 advantage。
2. V_w 不准时, δ_t 会带有 Critic bias。

8.5.3 Actor 怎样使用它

Actor 最大化:

$$J_{actor}(\theta) = \mathbb{E}_t [\log \pi_{\theta}(A_t \mid S_t) \hat{A}_t]$$

实现成最小化 loss:

$$L_{actor}(\theta) = -\mathbb{E}_t [\log \pi_{\theta}(A_t \mid S_t) \text{sg}(\hat{A}_t)]$$

这里也要 stop gradient:

- Actor 更新只改变“选这个动作的概率”。
- Critic/advantage 的参数由自己的价值目标更新。
- 否则 Actor loss 可能沿 $\hat{A}_t$ 反向修改 Critic, 混淆两个优化目标。

8.5.4 一次更新的数值例子

假设:

```
R_{t+1} = 2
gamma = 0.9
d_t = 0
V_w(s_t) = 5
V_w(s_{t+1}) = 6
```

一步 target:

$$Y_t^{(l)} = 2 + 0.9 \times 6 = 7.4$$

TD residual:

$$\delta_t = 7.4 - 5 = 2.4$$

这次动作结果比当前状态的预期高 2.4，因此：

- Actor 提高该动作概率。
- Critic 把 $V_w(s_t)$ 从 5 往 target 7.4 拉近。

如果 $d_t = 1$ ，则：

$$Y_t^{(1)} = R_{t+1} = 2$$

真正终止后没有环境定义的下一状态价值。

8.6 n-step Actor-Critic

8.6.1 为什么不一定只看一步

一步 TD 很快、方差低，但高度依赖 $V_w(S_{t+1})$ 。如果 Critic 早期很不准，Actor 会被带偏。

可以先观察 n 步真实 reward，再 bootstrap：

$$Y_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_w(S_{t+n})$$

如果 n 步内真正终止，终止后的 reward 和 bootstrap value 都不再加入。

对应的 n-step advantage estimate：

$$\hat{A}_t^{(n)} = Y_t^{(n)} - V_w(S_t)$$

8.6.2 n 控制什么

n 的大小	真实 reward 比例	Bootstrap 依赖	方差	Critic bias
小	少	强	较低	较强
大	多	弱	较高	较弱
到 episode 末尾	完整 G_t	无	最高	无 bootstrap bias

n-step 不是独立的新算法，而是 advantage/value target 的时间尺度选择。

8.6.3 截断不一定等于真正终止

并行采样常固定收集 T 步 rollout。到达 rollout 边界只表示“这批数据暂时截断”，不一定表示环境真正结束。

- 真正 terminal：后续价值为 0。
- 仅 rollout truncation：通常仍用 $V_w(S_{t+T})$ bootstrap。

如果把所有 rollout 边界都当 terminal，会系统性低估末端状态价值。

8.7 GAE：平滑组合不同时间尺度

8.7.1 从连续 TD Residual 出发

一步 advantage estimate 是：

$$\hat{A}_t^{(1)} = \delta_t$$

如果下一步也持续比预期好， δ_{t+1} 也应该回传到当前动作，只是影响要衰减：

$$\delta_t + \gamma \lambda \delta_{t+1}$$

继续累加得到 GAE：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

有限 rollout 中可从后往前递推：

$$\hat{A}_t = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}$$

8.7.2 GAE 与 n-step Advantage 的关系

GAE 也可理解为对不同 n-step advantage estimate 做指数加权：

$$\hat{A}_t^{GAE} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)}$$

因此它不是“另一个突然出现的 advantage 定义”，而是把一步、两步、三步等估计平滑组合起来。

8.7.3 λ 的含义

- $\lambda=0$ ： $\hat{A}_t = \delta_t$ ，退化为一歩 TD。
- λ 接近 1：纳入更长时间的 residual，更接近长回报。
- λ 越大：通常 bias 降低、variance 增大。
- λ 越小：通常 variance 降低、但更依赖 Critic 的短期 bootstrap。

实践中常见 $\lambda=0.95$ ，但它是需要结合任务 horizon、reward 噪声和 Critic 质量调节的超参数，不是理论固定值。

8.7.4 数值例子

假设连续三个 TD residual 为：

$$\delta_t = 2, \delta_{t+1} = 1, \delta_{t+2} = -1$$

令：

$$\gamma = 0.9, \lambda = 0.95, \gamma \lambda = 0.855$$

截断到三项：

$$\begin{aligned} \hat{A}_t^{GAE} &= 2 + 0.855 \times 1 + 0.855^2 \times (-1) \\ &\approx 2.124 \end{aligned}$$

当前动作先得到连续正面信号，但第三步出现负面 residual，因此最终 advantage 略高于 2，而不是简单只取第一步的 2。

8.7.5 Return Target 与 Advantage 的关系

PPO/A2C 实现中经常先算 GAE advantage，再构造 Critic target：

$$\hat{V}_t^{target} = \hat{A}_t + V_w(S_t)$$

原因是 advantage 定义本来就是：

$$A = Q - V$$

所以“value target = advantage estimate + old value estimate”是在把基线加回去。实现时通常保存 rollout 时的 value prediction，避免更新过程中 target 跟着当前网络反复变化。

8.8 Actor-Critic 的完整优化目标

8.8.1 Actor Loss

$$L_{actor} = -\mathbb{E}_t [\log \pi_{\theta}(A_t | S_t) \text{sg}(\hat{A}_t)]$$

它只负责：根据 advantage 的正负调整已采样动作概率。

8.8.2 Critic Loss

$$L_{critic} = \mathbb{E}_t [(V_w(S_t) - \text{sg}(\hat{V}_t^{target}))^2]$$

它只负责：让价值预测接近一步、n-step 或 GAE 构造出的 value target。

8.8.3 Entropy Bonus 属于哪里

Entropy 不是 REINFORCE 或 Actor-Critic theorem 必然推出来的项，而是额外加入的探索正则。

策略熵：

$$H(\pi_{\theta}(\cdot | s)) = -\sum_a \pi_{\theta}(a | s) \log \pi_{\theta}(a | s)$$

最小化形式可写成：

$$L_{entropy} = -\beta_H \mathbb{E}_t [H(\pi_{\theta}(\cdot | S_t))]$$

作用：

- 防止策略过早变成近乎确定性分布。
- 保留探索。
- 改善 early training 的状态覆盖。

它与第 10 章 SAC 的区别是：

- A2C/PPO 常把 entropy 当 regularization bonus。
- SAC 从最大熵目标重新定义 soft value 和 Bellman target，entropy 是算法核心目标的一部分。

8.8.4 总 Loss

共享网络实现常写：

$$L = L_{actor} + c_V L_{critic} + L_{entropy}$$

三个项的职责不同，不能因为写在一个式子里就认为它们来自同一推导。

8.8.5 共享网络与分离网络

Actor 和 Critic 可以：

- 共享 backbone，使用 policy head 和 value head。
- 使用完全分离的两个网络。

共享网络：

- 参数更少，表征可复用。
- Actor/Critic 梯度可能相互干扰。
- c_V 的尺度更敏感。

分离网络：

- 优化边界更清楚。
- 参数和计算更多。

8.9 混合动作空间中的 Actor-Critic

第 7 章把联合动作写成：

$$A_t = (K_t, U_t)$$

其中 K_t 是离散类型， U_t 是该类型下的连续参数。联合 log probability：

$$\log \pi_{\theta}(K_t, U_t | S_t) = \log \pi_{\theta_d}(K_t | S_t) + \log \pi_{\theta_c}(U_t | S_t, K_t)$$

使用 state-value Critic 时，Actor loss 为：

$$L_{actor} = -\mathbb{E}_t [(\log \pi_{\theta_d}(K_t | S_t) + \log \pi_{\theta_c}(U_t | S_t, K_t)) \text{sg}(\hat{A}_t)]$$

同一个 $\hat{A}_t$ 评价整个联合动作。

State-value Critic 的一个优点是只输入状态：

$$V_w(S_t)$$

它不需要枚举或显式表示复杂的联合动作空间。另一类方法会训练：

$$Q_w(S_t, K_t, U_t)$$

这种 Critic 能更直接区分离散类型和连续参数，但建模与优化更复杂。

工程上还要注意：

- 只计算实际选择的离散分支对应的连续 log probability。
- 分别监控 categorical entropy 和 continuous entropy，避免一项支配另一项。
- 对离散动作 mask 和连续参数边界使用一致的采样与 log-prob 计算。

8.10 A2C 与 A3C：采样组织方式

8.10.1 先区分 Framework 和 Algorithm

Actor-Critic 是框架：Actor 用 Critic 的估值信号更新。

A2C/A3C 是具体的数据采样和参数更新组织方式。它们不会重新定义 advantage，而是选择怎样并行产生 rollout、何时聚合梯度。

8.10.2 A3C

A3C 是 Asynchronous Advantage Actor-Critic：

1. 多个 worker 各自持有环境副本。
2. 每个 worker 用本地策略采样短 rollout。

3. 计算 n-step return 或 advantage。
4. 异步把梯度应用到共享全局网络。
5. 再从全局网络同步最新参数。

优点：

- 不同 worker 提高状态覆盖和数据多样性。
- 在不使用 replay buffer 的 on-policy 场景中降低样本相关性。

问题：

- 异步更新存在参数陈旧和 nondeterminism。
- 工程调试较复杂。

8.10.3 A2C

A2C 可理解为同步版本：

1. N 个环境使用同一版策略并行采样 T 步。
2. 聚合成 $N \times T$ 的 rollout batch。
3. 统一计算 advantage 和 value target。
4. 对 Actor/Critic 做一次或若干次同步更新。

A2C 通常更适合现代 GPU batch 计算，结果也更容易复现。

维度	A2C	A3C
Worker 更新	同步聚合	异步写共享参数
参数版本	一批数据基本一致	Worker 可能稍陈旧
GPU 利用	更自然	相对困难
可复现性	较好	较弱
核心 advantage	TD / n-step / GAE 均可	TD / n-step 为经典做法

8.11 一轮训练的完整数据流

1. 用当前 policy π_{θ} 在多个环境采样 T 步。
2. 保存 state、action、reward、done、log probability、value estimate。
3. 在 rollout 末端：


```
terminal -> bootstrap value = 0
truncation -> bootstrap value = V_w(last_state)
```
4. 从后往前计算 TD residual。
5. 用 n-step 或 GAE 得到 $A_{\hat{t}}$ 。
6. 构造 value target = $A_{\hat{t}}$ + rollout-time value estimate。
7. Actor 最小化 $-\log_{\text{prob}} * \text{stop_gradient}(A_{\hat{t}})$ 。
8. Critic 回归 $\text{stop_gradient}(\text{value target})$ 。
9. 可选加入 entropy bonus。
10. 更新后丢弃或谨慎限制这批 on-policy rollout 的复用。

这条数据流是理解第 9 章 PPO 的前置输入。PPO 不会重新发明 Critic 或 GAE，而是重点改变第 7 步的 Actor objective，并允许对同一 rollout 做有限的多 epoch 更新。

8.12 本章例子与对比

8.12.1 REINFORCE 与 Actor-Critic

维度	REINFORCE with Baseline	Actor-Critic
Baseline/Value 标签	完整 G_t	TD、n-step 或 GAE target
Bootstrap	否	通常是
更新等待	通常到 episode 结束	短 rollout 后即可
方差	较高	较低
估计偏差	无 bootstrap bias	有 Critic bias
学习模块	Policy + 可选 MC baseline	Actor + TD Critic

8.12.2 游戏 Agent

- Actor 输出移动、攻击、防御的动作分布。
- Critic 估计当前局面的 $V_w(s)$ 。
- 如果一次动作得到 $\delta_t > 0$ ，说明结果超出当前局面平均预期，Actor 提高该动作概率。
- GAE 会把后面数步持续发生的正负 TD residual 传回当前动作。

8.12.3 为什么 Critic 不直接决定动作

在 state-value Actor-Critic 中：

- Critic 只输出 $V_w(s)$ ，不负责在动作中取 argmax。
- Actor 输出动作分布并真正执行。
- Critic 的职责是提供相对学习信号，而不是替代 Actor。

这与 DQN 的 Q 网络直接通过 $\arg \max_a Q(s,a)$ 选动作不同。

8.13 本章面试高频问题

8.13.1 Actor 和 Critic 分别做什么？

Actor 输出并更新策略；Critic 估计状态或动作价值，为 Actor 构造较低方差的 advantage estimate。

8.13.2 TD Residual 为什么能估计 Advantage？

因为在真实 V_π 下：

$$\mathbb{E}[\delta_t | s_t, a_t] = Q_\pi(s_t, a_t) - V_\pi(s_t) = A_\pi(s_t, a_t)$$

实际用近似 V_w 时还会带有 Critic 误差。

8.13.3 n-step 与 GAE 的关系？

n-step 选择一个固定 bootstrap horizon；GAE 对多个 n-step advantage estimate 做指数加权，避免只押注一个时间尺度。

8.13.4 为什么 Actor Loss 要对 Advantage Stop Gradient？

Advantage 是提供给 Actor 的样本权重。Actor loss 应只更新策略概率，Critic 应由自己的 value loss 更新；否则两类目标的梯度会意外耦合。

8.13.5 A2C 和 A3C 区别？

A2C 同步聚合多个环境的 rollout 后更新；A3C 让多个 worker 异步更新共享参数。它们共享 Actor-Critic 基础，主要区别是并行与参数同步方式。

8.13.6 Entropy Bonus 是 Actor-Critic 必需的吗？

不是。它是常用探索正则。去掉它仍然是 Actor-Critic，但策略可能更快变得确定、探索不足。

8.14 本章练习

1. 从 $A_\pi = Q_\pi - V_\pi$ 推导为什么一步 TD residual 可估计 advantage。
2. 对给定的 $R_{t+1}=1$ 、 $\gamma=0.9$ 、 $V(s_t)=4$ 、 $V(s_{t+1})=5$ ，计算一步 target 和 TD residual。
3. 写出三步 value target 和对应 advantage estimate。
4. 解释 GAE 中 $\lambda=0$ 与 $\lambda \rightarrow 1$ 的含义。
5. 说明 terminal 与 rollout truncation 在 bootstrap mask 上为什么不同。
6. 写出混合动作 (k,u) 的 Actor loss。
7. 按数据流解释 A2C 一轮 rollout 到更新的全过程。

▼ 参考答案（点击展开）

1. **TD residual 与 advantage**: $Q_\pi(s_t, a_t) = \mathbb{E}[R_{t+1} + \gamma(1-d_t)V_\pi(S_{t+1}) | s_t, a_t]$ 。减去 $V_\pi(s_t)$ 后，得到 $\mathbb{E}[\delta_t | s_t, a_t] = A_\pi(s_t, a_t)$ 。用一次 transition 和近似价值 V_w 时， δ_t 是带采样噪声和 Critic bias 的估计。
2. **数值题**: 未终止时 $Y_t^{(1)} = 1 + 0.9 \times 5 = 5.5$ ， $\delta_t = 5.5 - 4 = 1.5$ 。结果高于当前状态预期，Actor 应提高已采样动作概率。
3. **三步目标**: $Y_t^{(3)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V_w(S_{t+3})$ ；对应 $\hat{A}_t^{(3)} = Y_t^{(3)} - V_w(S_t)$ 。若三步内真正终止，终止后的项和 bootstrap value 为 0。
4. **λ 两端**: $\lambda=0$ 时 GAE 只保留 δ_t ，等于一步 TD advantage； λ 趋近 1 时更长时间的 residual 权重变大，在 episodic 且边界处理正确时更接近 Monte Carlo advantage。前者通常方差低、bias 高，后者相反。
5. **Terminal vs truncation**: terminal 表示 MDP 真正结束，环境定义的未来 return 为 0；truncation 只是采样批次或时间上限结束，真实过程可能继续，因此通常需要用最后状态的 V_w bootstrap。
6. **混合动作 loss**: 若 $\pi(k, u | s) = \pi_d(k | s) \pi_c(u | s, k)$ ，则 $L_{actor} = -[\log \pi_d(k_t | s_t) + \log \pi_c(u_t | s_t, k_t)] \text{sg}(\hat{A}_t)$ 。只计算实际选择的离散分支对应的连续参数 log probability。
7. **A2C 数据流**: 多个环境用同一策略同步采样 T 步，保存动作 log probability 和 value；按 terminal/truncation 决定末端 bootstrap；反向计算 TD residual 和 GAE；构造 value target；Actor 用 $-\log \pi \hat{A}$ 更新，Critic 回归 value target，可选加入 entropy bonus；完成后再用新策略采下一批数据。